data.frame : 12 obs of 5 variables:

$ Temperature : num 25 30 28 35 22 27 33 29 32 24 ...

$ Humidity : num 80 60 75 55 85 78 50 70 65 88 ...

$ Rainfall : num 120 40 90 30 150 100 20 85 60 140 ...

$ Soil_moisture : Factor w/ 3 levels "High","Low","Medium": 1 2 3 2 1 3 2 3 2 1 ...

$ DiseaseRisk : Factor w/ 3 levels "High","Low": 1 2 1 2 1 2 2 1 2 1 ...

Training rows : 8

Testing rows : 4.

Ex:No: 4A Date: 26/8/26	**Predicting Crop Disease Risk Using Decision Tree Algorithm**

Problem Description:

An agricultural research institute monitors environmental conditions to reduce crop losses caused by plant diseases. Factors such as temperature, humidity, rainfall, and soil moisture strongly influence disease occurrence. The institute wants to **predict whether crops are at high or low disease risk** based on these conditions using a **Decision Tree model** enabling early preventive measures.

Objective:

To build and evaluate a Decision Tree classification model in R to predict crop disease risk using environmental data with train–test split and accuracy calculation.

Procedure:

- **Step 1: Create the Crop Environment Dataset - crop_data**

	Temperature	Humidity	Rainfall	SoilMoisture	DiseaseRisk
1	25	80	120	High	High
2	30	60	40	Medium	Low
3	28	75	90	High	High
4	35	55	30	Low	Low
5	22	85	150	High	High
6	27	78	100	High	High
7	33	50	20	Low	Low
8	29	70	85	Medium	High
9	31	65	60	Medium	Low
10	24	88	140	High	High
11	34	52	25	Low	Low
12	26	82	110	High	High

```
crop_data <- data.frame(
  Temperature = c(25, 30, 28, 35, 22, 27, 33, 29, 31, 24, 34, 26),
  Humidity = c(80, 60, 75, 55, 85, 78, 50, 70, 65, 88, 52, 82),
  Rainfall = c(120, 40, 90, 30, 150, 100, 20, 85, 60, 140, 25, 110),
  Soil_moisture = c("High", "Medium", "High", "Low", "High", "High",
                    "Low", "Medium", "Medium", "High", "Low",
                    "High"),
  disease_risk = c("High", "Low", "High", "Low", "High", "High", "Low",
                   "High", "Low", "High", "Low", "High")
)
#Print crop_data
```

	Temperature	Humidity	Rainfall	SoilMoisture	DiseaseRisk
1	25	80	200	High	High
2	30	60	40	Low	Low
3	28	75	90	Medium	High
4	35	55	30	Low	Low
5	22	85	150	High	High
6	27	78	800	medium	High
7	33	50	20	Low	Low
8	29	70	85	medium	High
9	31	65	60	Low	Low
10	24	88	140	High	High
11	26	78	110	medium	High
12	32	68	70	Low	Low.

- **Step 2: Convert Categorical Variables to Factors**

```
crop_data$soil_moisture <- as.factor(crop_data$soil_moisture)
crop_data$disease_risk <- as.factor(crop_data$disease_risk)
```

```
# str() - crop_data
str(crop_data)
```

Step 3: Train–Test Split – 70% training and 30% Testing

```
index <- sample(1:nrow(crop_data), 0.7*nrow(crop_data))
train_data <- crop_data[index, ]
test_data <- crop_data[-index, ]
```

```
#Print number of rows in train and test data
cat("Training rows:", nrow(train_data), "\n")
cat("Testing rows:", nrow(test_data), "\n")
```

- **Step 4: Build the Decision Tree Model - disease_tree**

```
library(rpart.plot)
library(rpart)
disease_tree <- rpart(disease_risk ~ Temperature + Humidity + Rainfall + SoilMoisture,
data = train_data, method = "class",
control = rpart.control(minsplit = 0, cp=0))
```

>disease_tree

n=8

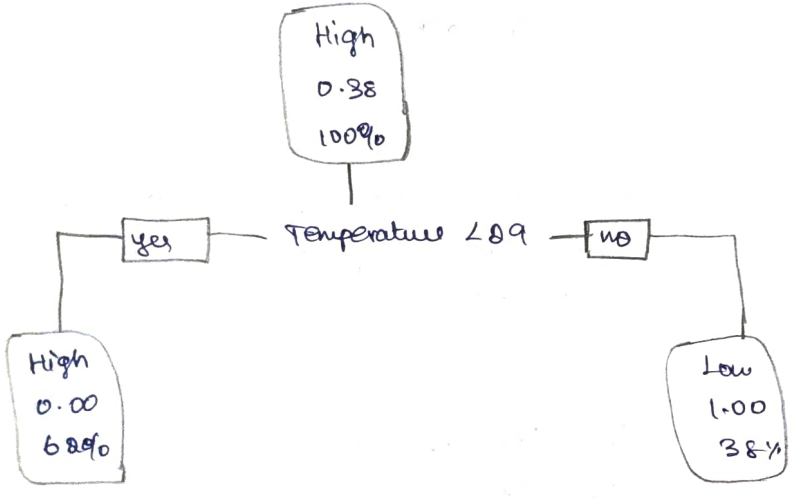
node, split, n, loss, yval, (yprob)

* denotes terminal node.

1) root 8 3 High (0.6250000 0.3750000)

2) Temperature < 89 5 0 High (1.0000000 0.0000000) *

3) Temperature >= 89 3 0 Low (0.0000000 1.0000000) *



>Predicted_risk

1 7 8 9

High Low Low Low

Levels: High Low

> confusion_matrix

Actual	predicted High	Low
High	1	1
Low	0	8

> Accuracy

0.75

#Print disease_tree

print(disease_tree)

- **Step 5: Visualize the Tree**

rpart.plot(disease_tree)

- **Step 6: Predict Disease Risk**

Predicted_risk <- predict(disease_tree, test_data, type="class")

Predicted_risk

- **Step 7: Confusion Matrix**

confusion_matrix <- table(Actual = test_data$Disease_risk, Predicted = predicted_risk)

confusion_matrix.

- **Step 8: Accuracy Calculation**

accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matri

accuracy.

Result:

Thus, the program successfully demonstrates the use of the **Decision Tree algorithm in R** to predict crop disease risk based on environmental conditions. The model effectively classifies disease risk using a **train–test split and accuracy evaluation**, providing valuable insights for early agricultural intervention.

>head (iris)

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.8	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.8	setosa
3	4.7	3.8	1.3	0.8	setosa
4	4.6	3.1	1.5	0.8	setosa
5	5.0	3.6	1.4	0.8	Setosa
6	5.4	3.9	1.7	0.4	Setosa.

Ex:No: 4B Date: 26/3/26	**Decision Tree Algorithm Using Iris Dataset**

Problem Description:

The Iris dataset contains measurements of iris flowers belonging to three species: *Setosa*, *Versicolor*, and *Virginica*. The goal is to use flower measurements such as sepal length, sepal width, petal length, and petal width to **classify the species of iris flowers** using a **Decision Tree algorithm**.

Objective:

To implement a Decision Tree classification model in R using the Iris dataset, perform train–test splitting, predict flower species, and evaluate model accuracy.

Procedure:

→ **Step 1: Install and Load Necessary Libraries**

- Install and load rpart (for decision tree modeling) and rpart.plot (for visualization).
- Load caret for dataset splitting and evaluation.

```
library (rpart)
library (rpart.plot)
library (caret)
```

- **Step 2: Load and Explore the Dataset**
  - Use the built-in iris dataset.
  - Print first six rows of iris dataset.

```
data (iris)
Print ("First few rows of dataset :")
Print (head (iris))
```

Training rows : 120

Testing rows : 80

> tree_model

n=120

node), split, n, loss, yval, (yprob)

* denotes terminal node

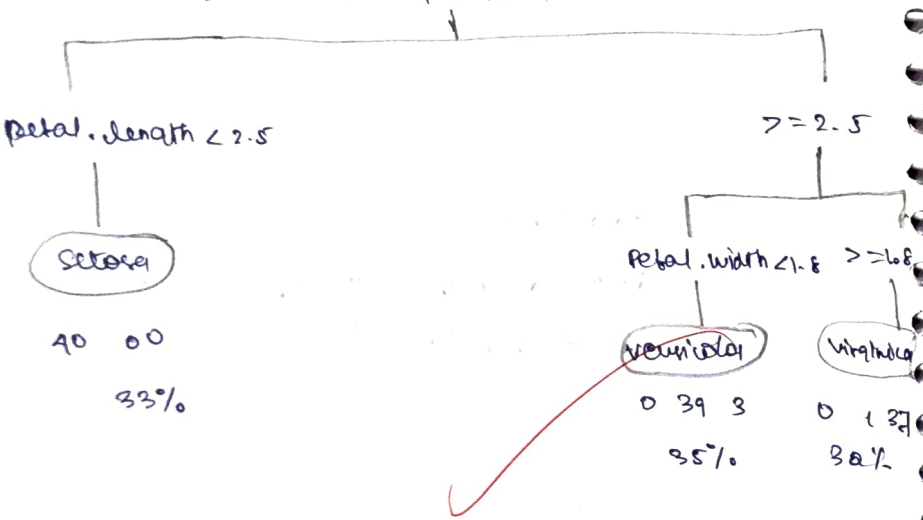
1) root 120 80 Setosa (0.33333333 0.33333333 0.33333333)

2) Petal.Length < 2.45 40 0 Setosa (1.00000000 0.00000000 0.00000000) *

3) Petal.Length >= 2.45 80 40 Versicolor (0.00000000 0.50000000 0.50000000) *

4) Petal.Width < 1.75 42 3 Versicolor (0.00000000 0.92857143 0.07142857) *

5) Petal.Width >= 1.75 38 1 Virginica (0.00000000 0.02631579 0.97368421) *



- **Step 3: Split the Dataset**
  - **80% for training, 20% for testing** using createDataPartition()

```
train_index <- createDataPartition(iris$Species, p=0.8, list=FALSE)
train_data <- iris[train_index,]
test_data <- iris[-train_index,]
cat("Training rows: ", nrow(train_data), "\n")
cat("Testing rows: ", nrow(test_data), "\n")
```

- **Step 4: Train the Decision Tree Model**
  - Use the rpart() function to build the decision tree and print it.

```
tree_model <- rpart(Species~., data = train_data,
                    method = "class")
print(tree_model)
```

- **Step 5: Visualize the Decision Tree**
  - Use rpart.plot() to plot the decision tree structure.

```
rpart.plot(tree_model, main = "Decision Tree for Iris
Dataset", type=3, extra=101, under=TRUE,
```

Confusion Matrix :

Confusion Matrix and Statistics.

Prediction	Reference Setosa	Versicolor	Virginica.
Setosa	10	0	0
Versicolor	0	10	0
Virginica	0	0	8

Overall Statistics

Accuracy : 0.9333

95% CI : (0.7793, 0.9918)

NO Information Rate : 0.3333

P-value [Acc > NIR] : 8.747e-12

Kappa : 0.9

Mcnemar's Test P-value : NA

Statistics by class :

	class: setosa	class: versicolor	class: virginica
sensitivity	1.0000	1.0000	0.8000
specificity	1.0000	0.9000	1.0000
Pos Pred value	1.0000	0.8333	1.0000
Neg Pred value	1.0000	1.0000	0.9091

:

:

> accuracy

Accuracy	kappa	AccuracyLower	AccuracyUpper	Accuracy Null
9.333333e-01	9.000000e-01	7.792646e-01	9.910219e-01	3.333333e-01

- **Step 6: Evaluate the Model**
  - Predict on the test dataset.
  - Compute the **confusion matrix** and accuracy score.

Make Predictions on Test Data

Predictions ← predict(tree_model, test_data, type = "class")

Evaluate Model Performance Confusion Matrix and Accuracy

conf_matrix ← confusionMatrix(Predictions, test_data$ Species)

Cat("Confusion Matrix : \n")

Print(conf_matrix)

Cat("Accuracy of Decision Tree Model :", conf_matrix$overall['Accuracy'],"\n

Result:

The Decision Tree model was successfully implemented and visualized in R using the iris dataset. The model achieved an accuracy of 93.33%, showing high classification performance. The decision tree structure was visualized, helping in understanding how the algorithm makes predictions based on feature splits.